A D PATEL INSTITUTE OF TECHNOLOGY

Department of Information Technology

AI-DRIVEN ACADEMIC TIMETABLE GENERATOR & SCHEDULE OPTIMIZER

PROJECT REPORT

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology in Information Technology
Semester V

Subject: Programming with Python

Submitted by

Desai Aryan M.
(12402080601009)

Faculty Guide

Prof. Nayankumar Mali
Assistant Professor

Academic Year 2026–2027

New Vallabh Vidyanagar, Vitthal Udyognagar – 388121
Anand, Gujarat, India

CERTIFICATE

This is to certify that the project report entitled **“AI-Driven Academic Timetable Generator & Schedule Optimizer”** has been prepared by **Desai Aryan M. (12402080601009)**, student of Bachelor of Technology in Information Technology, Semester V, at A D Patel Institute of Technology. The work is submitted in partial fulfilment of the academic requirements for the subject **Programming with Python**.

The project demonstrates the use of programming logic, data structures, scheduling algorithms and a Python backend to support the generation and validation of academic timetables. The report has been prepared for academic evaluation under the guidance of the faculty member.

Date: _____

Place: New Vallabh Vidyanagar

_____	_____
Lab Teacher	Head of Department Dr. Priyang Bhatt

ACKNOWLEDGEMENT

I express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for giving me the opportunity to complete this project as part of the Programming with Python subject.

I am thankful to my faculty guide for valuable guidance, suggestions and support during the planning and development of the project. This work helped me understand how programming concepts can be applied to a practical institutional problem. In particular, I gained experience with structured data, scheduling logic, conflict checking, testing and the design of a simple web-based interface.

I also thank my classmates and everyone who provided assistance or encouragement during the project. Finally, I am grateful to my family for their continued support.

Desai Aryan M.
(12402080601009)

CONTENTS

Section	Page
Certificate	2
Acknowledgement	3
Abstract	4
1. Introduction	5
2. Background and Algorithm	6
3. System Design	7
4. Implementation	8
5. Testing and Results	9
6. Conclusion and References	10

ABSTRACT

Preparing a college timetable manually is time-consuming because classrooms, laboratories, teachers, subjects and student divisions must be coordinated without clashes. This project presents a web-based Academic Timetable Generator designed around the academic data of A D Patel Institute of Technology. It supports eight departments, 36 rooms and 55 faculty members, and generates timetables for student divisions across Monday to Friday.

The interface uses HTML, CSS/Tailwind CSS and JavaScript. A Python backend using the standard library provides REST API endpoints for data access, generation, validation and CSV export. The scheduler assigns faculty using priority scores, schedules laboratories in consecutive periods, places parallel electives in separate rooms and uses booking sets to prevent double booking. Testing recorded zero faculty and room clashes in three browser-engine runs, with generation taking approximately 8–16 milliseconds in Node.js. The system can be improved by reducing free periods and adding persistent database storage.

1. INTRODUCTION

1.1 Background

A college timetable coordinates the time, classroom and teacher for every lecture and practical session. In a multi-department institute, rooms and faculty are shared, laboratories have limited capacity, and elective subjects may need to run simultaneously. Preparing schedules manually can lead to repeated work and accidental clashes.

1.2 Problem Statement

The project aims to create a system that generates weekly timetables while respecting room availability, faculty availability, laboratory duration, elective arrangements and institutional working days. A teacher or room must not be assigned to two classes in the same time slot.

1.3 Objectives

- Organize rooms, departments, divisions, subjects and faculty in structured data.
- Generate timetables for all departments in one run and prevent double booking.
- Allocate faculty according to subject priorities, skills and workload.
- Display class and faculty timetables, including workload information.
- Allow timetable editing with conflict warnings and export to PDF/print, PNG and CSV.

1.4 Scope

The project covers Semester V for eight departments: Computer Engineering, Information Technology, Computer Science & Design, Artificial Intelligence & Data Science, Mechanical, Civil, Electrical and Chemical Engineering. The current prototype keeps institutional data in source code and does not include user login or database persistence.

2. BACKGROUND AND ALGORITHM

2.1 Timetabling as a Constraint Problem

A Constraint Satisfaction Problem (CSP) contains variables, possible values and rules that restrict valid combinations. In this project, timetable cells are assigned subjects, faculty and rooms. Constraints include one class per faculty member at a time, one class per room at a time, consecutive periods for laboratories and separate rooms and teachers for parallel electives.

2.2 Scheduling Approach

The scheduler uses a greedy approach with constraint checks. It places activities in a planned order and skips any placement that would create a clash. This is fast and straightforward, but may leave some periods unfilled. Backtracking or metaheuristic methods could explore more alternatives but would require additional computation and implementation complexity.

2.3 Main Algorithm Steps

- Load departments, faculty, curriculum, rooms and time-slot information.
- Calculate the number of divisions from student intake.
- Select suitable faculty for subjects using priority and workload scores.
- Place laboratory sessions in consecutive two-period blocks.
- Schedule parallel elective options at the same time in different rooms.
- Place regular lectures only when the faculty member and room are available.
- Mark unfilled cells as free lectures and allow users to review or edit the result.

2.4 Data Structures

Arrays and structured objects store institutional information. Maps associate subject codes with assigned faculty, while sets record booked rooms and faculty for each day and period. Timetable grids store the final schedule, and workload structures summarize assigned teaching hours. Set lookups support fast conflict checks.

3. SYSTEM DESIGN

3.1 System Modules

Module	Purpose
Room Entry	Maintains classrooms and laboratories, including capacity and features.
Student Entry & Divisions	Stores intake and calculates the number of divisions.
Subjects & Credits	Organizes curriculum, subject categories and credit details.
Faculty Entry Detail	Stores faculty profiles, skills and subject priorities.
Faculty Timetable & Workload	Displays faculty schedules and workload summaries.
Student Timetable	Generates, edits and exports division timetables.

3.2 Weekly Structure

The timetable runs from Monday to Friday, with eight teaching periods per day and a lunch break between morning and afternoon sessions. The listed time slots are: 9:05–9:55, 10:00–10:50, 10:55–11:45, 11:50–12:40, 1:25–2:15, 2:20–3:10, 3:15–4:05 and 4:10–5:00. Saturday and Sunday are holidays.

3.3 System Workflow

- Open the web page and load the institutional data.
- Review or add room and faculty information where supported.
- Generate the timetable and select a department, division or faculty member.
- Edit a period if required; the system checks for faculty and room conflicts.
- Export the final schedule in the required format.

3.4 Architecture

The browser version performs scheduling in JavaScript. An optional Python server exposes REST endpoints that return data and results as JSON. The backend includes modules for configuration, data models, institutional data, scheduling, server routes and tests.

4. IMPLEMENTATION

4.1 Technologies Used

Technology	Use
HTML5	Page structure, tabs, tables, forms and dialogs.
CSS / Tailwind CSS	Responsive layout, styling and print formatting.
JavaScript (ES6)	Institution data, scheduling logic, page interaction and timetable rendering.
Canvas API / Blob API	Creates downloadable PNG and CSV files.
Python 3 standard library	Provides backend models, scheduler, REST API and tests.
Node.js	Runs the browser scheduling logic for verification.

4.2 Python Backend

The optional backend uses Python's built-in http.server module, so it does not require additional packages. Its REST API includes endpoints to check service health, list departments, retrieve faculty and curriculum, list rooms, view workload, generate a timetable, validate a manual slot change and export a timetable as CSV.

4.3 Editing and Export

A user can open a timetable cell and change the subject, faculty or room. The application checks the same day and time across other divisions and warns if a faculty member or room is already booked. Print/PDF uses the browser's print layout, PNG export draws the timetable on a canvas, and CSV export prepares a downloadable file in memory.

4.4 Limitations

- Room, faculty and curriculum data are stored in source code rather than a database.
- The prototype does not provide login, roles or integration with student records.
- The greedy scheduler can leave free periods when no valid placement is found.
- Actual scheduled faculty hours may not match the displayed standardized workload targets.

5. TESTING AND RESULTS

5.1 Verification

The Python test suite and the JavaScript scheduling engine were used to verify timetable generation. The browser engine was run with three different seeds, and generated grids were checked for room and faculty clashes. The table below summarizes selected results reported during testing.

Test	Expected result	Observed result
Faculty double booking	No clashes	None found
Room double booking	No clashes	None found in three browser runs
Saturday schedule	No Saturday periods	None
Parallel electives	Different rooms and faculty	Verified
Laboratory timing	Consecutive periods without overlap	Verified
Browser timetable output	11 division timetables	11 timetables; 440 period cells
Generation speed	Under one second	About 8–16 ms in Node.js
Faculty workload target	Most staff at 12–16 hours	8 of 55 in range; average 8.4 scheduled hours

5.2 Discussion

The key result is that the generated browser timetables had no room or faculty clashes across the three verification runs. The scheduling process completed in milliseconds, making repeated generation practical. However, approximately 28% of the 440 period cells were free lectures. Workload testing also showed that only 8 of 55 faculty members were within the 12–16 hour range when calculated from actual scheduled periods. These results indicate that conflict prevention works well, while period utilization and workload balancing need further improvement.

5.3 Future Improvements

- Use backtracking or a metaheuristic to reduce the number of free periods.
- Improve workload balancing within the scheduling algorithm itself.
- Add a database, authentication and role-based access.
- Import institutional data from spreadsheets and add automated interface tests.

6. CONCLUSION AND REFERENCES

6.1 Conclusion

The Academic Timetable Generator demonstrates how programming and scheduling algorithms can simplify a complex college task. It organizes institutional data, assigns faculty, schedules practicals and electives, prevents room and faculty clashes, and provides editable timetables with export options. The Python backend adds a structured way to access and validate timetable data through REST API endpoints. Verification showed fast generation and no clashes in the tested browser runs. The main areas for improvement are reducing free periods, improving actual workload balance, and introducing database storage and user authentication.

6.2 References

1. Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson.

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. *Introduction to Algorithms*. MIT Press.

3. Burke, E. K. and Petrovic, S. “Recent research directions in automated timetabling.” *European Journal of Operational Research*, 140(2), 2002.

4. MDN Web Docs: HTML, CSS, JavaScript and browser APIs. https://developer.mozilla.org

5. Tailwind CSS Documentation. https://tailwindcss.com/docs

6. Python Documentation: http.server. https://docs.python.org/3/library/http.server.html

7. Project source files: index.html and the Python backend modules (config.py, models.py, database.py, csp_solver.py, server.py and main.py).

— End of Report —